



Public Healthcare AIDP Workshop

Execution team guide for a public healthcare lakehouse workshop using Oracle AI Data Platform, Autonomous AI Lakehouse, Oracle Analytics Cloud, spatial analytics, document chat, and optional real-time labs.

Core labs 3 hours	Gold model Snowflake schema
Raw formats CSV, JSON, GeoJSON, DOCX	Privacy Synthetic data only



Contents

About This Workshop

Business Use Case

Workshop Info

Architecture

Workshop Role Convention

Who Does What

Lab 1 - Create a Healthcare Lakehouse with AI Data Platform

Files to upload

Admin Steps

Participant Steps

Lab 2 - Build Bronze and Silver Layers with AI Data Platform

Admin Notes

Participant Steps

Bronze Layer in AIDP

Silver Layer in AIDP

Shared Bronze-to-Silver teaching pattern

Gold Layer in Autonomous AI Lakehouse

Gold branch used in the workshop

Gold Dimensions

Gold Facts

Suggested Transformations

Lab 3 - Instructor-Led Claims star schema and OAC Workbook

Admin Steps

Participant Steps

Instructor-led Claims star schema build

Recommended OAC canvases for the instructor-led workbook

Lab 4 - DIY Facility Access Daily star schema and Participant Dashboard Challenge

Admin Notes

Participant Steps

DIY Build Target

Participant Challenge Prompts

Generative Analytics Prompts

Optional Lab 5 - Claims Denial Risk Scoring

Admin Preparation

Participant Steps

Optional Lab 6 - Claims and Policy Copilot

Admin Preparation

Participant Steps

Optional Lab 7 - Governance and Operationalization

Admin Steps

Success Criteria

Public Healthcare AIDP Workshop

About This Workshop

Public healthcare organizations need to balance service access, clinical capacity, staffing, immunization campaigns, claims adjudication, public-program disbursement, membership eligibility, provider accreditation, and emerging public-health risks. In this workshop, you will build a lakehouse analytics pipeline for Metro Public Health Authority (MPHA), a fictional public healthcare network serving five districts through hospitals, urgent-care centers, and community clinics.

You will load synthetic healthcare operations data, refine it through a medallion architecture using Spark and Delta Lake in Oracle AI Data Platform, and serve the Gold business layer through dimensional stars in Autonomous AI Lakehouse. The workshop then branches into two outcomes: an instructor-led Claims star schema with a guided OAC workbook, and a participant-built Facility Access Daily star schema used to test whether participants can model and visualize the same pattern on their own.

Business Use Case

MPHA leadership wants a daily and weekly decision-support view that answers:

- Which facilities are exceeding wait-time or occupancy targets?
- Which districts have the highest access risk?
- Are immunization campaigns reaching older residents and high-need districts?
- Is respiratory illness pressure creating emergency department strain?
- Which service-quality issues are growing and how quickly are they being resolved?
- Which claim types or programs have rising denial rates or payment leakage?
- Are public healthcare disbursements timely, paid, and traceable by funding source?
- Which membership segments need eligibility renewal or high-risk care management?
- Which providers have accreditation gaps, corrective actions, or expiry risk?
- Where should leaders prioritize staffing, clinic sessions, outreach, and capacity relief?

Workshop Info

Length: 3 hours for core labs; optional extensions add 45 to 60 minutes each.

Services used:

- Oracle Object Storage
- Oracle AI Data Platform with Spark and Delta Lake
- Autonomous AI Data Lakehouse or Autonomous Data Warehouse
- Oracle Analytics Cloud
- Optional: OCI Generative AI Agents

Dataset period: January 1, 2025 to June 30, 2025

Dataset type: Five synthetic CSV datasets, one JSONL event dataset, one GeoJSON spatial dataset, and one synthetic operating playbook document

Architecture

1. Raw CSV, JSONL, GeoJSON, and document files are uploaded to object storage as the immutable landing zone.
2. AI Data Platform creates the **Bronze** layer as schema-preserving Delta tables with ingestion metadata.
3. AI Data Platform creates the **Silver** layer as conformed Delta tables with typed fields, quality rules, reference joins, spatial feature extraction, document chunk metadata, and derived operational flags.
4. Autonomous AI Lakehouse hosts the **Gold** layer as two teaching paths: an instructor-led Claims star schema and a DIY Facility Access Daily star schema, both built from the shared Silver foundation.
5. OAC connects only to the Claims star schema during the guided dashboard lab; participants design their own Facility Access Daily dashboard afterward as the hands-on challenge.

Workshop Role Convention

To keep execution responsibilities clear during the workshop:

- **Admin step** means the facilitator, environment owner, or platform administrator performs the action once for the shared workshop environment.
- **Participant step** means each participant or team performs the action in their own hands-on flow.

Who Does What

Admin / facilitator responsibilities

- provision shared AIDP, AI Lakehouse, and OAC resources
- create shared users, connections, catalogs, and datasets
- preload any assets or datasets that should not be repeated by every participant
- enable assistant, governance, or optional shared services where needed

Participant responsibilities

- sign in to the prepared environment
- run notebooks and validate outputs
- load or validate their own workshop data where requested
- build workbook content and complete guided or DIY exercises
- test optional ML and agent labs when included

Lab 1 - Create a Healthcare Lakehouse with AI Data Platform

Objectives

- Provision or identify the Oracle AI Data Platform environment for the workshop.
- Create the workspace, Spark cluster, and catalogs needed for the healthcare flow.
- Create or identify an object storage bucket for workshop data.

- Upload the raw MPHA CSV files.
- Create a Spark notebook in AI Data Platform.
- Read and profile the raw files.

Files to upload

Upload these five CSV files from `data/raw/`:

- `district_health_profile.csv`
- `facility_provider_master.csv`
- `facility_operations_daily.csv`
- `population_health_weekly.csv`
- `claims_membership_disbursement.csv`

Upload these additional source files:

- `data/raw_json/facility_capacity_events.jsonl`
- `data/raw_spatial/healthcare_service_areas.geojson`
- `documents/MPHA_Winter_Respiratory_Response_Playbook.docx`

Admin Steps

Participants: you can skip this section and join once the facilitator confirms the shared environment is ready.

1. Provision or confirm the Oracle AI Data Platform instance.

- Open **Analytics & AI**, then **AI Data Platform**. - Click **Create AI Data Platform** if the shared workshop environment does not already exist. - Use workshop-friendly names such as `mpha-aidp` for the instance and `mpha-workshop-ws` for the default workspace. - Choose **Standard** for simpler workshop setup, or **Advanced** if your tenancy requires finer policy control. - Apply the generated policy statements in the correct tenancy or compartment, then complete provisioning.

2. Assign workshop access using AIDP roles.

- Open **Roles**. - Grant facilitator accounts the admin or workspace-admin role required for setup. - Grant participant accounts builder-level access for notebooks, workspace usage, and catalog access. - Validate that a facilitator can see the workspace, catalogs, and compute choices.

3. Create the shared workshop workspace if you are not using the default one.

- Open **Workspaces** and click **Create**. - Use a name such as `mpha-core-workshop`. - Set the default catalog to the shared standard catalog that will hold the healthcare landing volumes.

4. Create the shared Spark compute cluster.

- Open **Compute** and click **Create**. - Use a name such as `mpha-spark-cluster`. - Select the Spark runtime. - If available, start with **Quickstart**. - Size the driver and workers for workshop execution rather than production-scale tuning. - Enable autoscaling if your team prefers elastic usage.

5. Create the standard catalog and landing schema for Object Storage-backed data.

- Open **Master Catalog**. - Create a standard catalog such as `MPHA_WORKSHOP_CAT`. - Create a schema such as `mpha_landing`.

6. Create the external volumes used by the workshop.

- In the `mpha_landing` schema, create external volumes for: - `mpha/raw/` - `mpha/raw_json/` - `mpha/raw_spatial/` - `mpha/documents/` - `mpha/bronze/` - `mpha/silver/` - `mpha/gold_stage/` - `mpha/vector/` - For each one, open the schema Volume tab, click Create Volume, choose External, and point it to the correct compartment, bucket, and folder.

7. Create the external Autonomous AI Lakehouse catalog for Gold publishing and validation.

- In Master Catalog, click Create Catalog. - Set Catalog Type to External. - Choose the connection type for Autonomous AI Lakehouse. - Use either wallet-based access or direct instance-based connectivity, depending on your environment. - Test and save the catalog with a name such as `MPHA_AILH_CAT`.

8. Set up the shared notebook structure in AIDP.

- Create folders such as `01_bronze`, `02_silver`, `03_gold`, `04_ml`, `05_agent`, and `shared_assets`. - Upload or create the starter notebooks: - `aidp_bronze_pyspark.py` - `aidp_silver_pyspark.py` - `aidp_gold_pyspark.py`

Participant Steps

1. Sign in to the workshop workspace and confirm that you can see the shared folders, catalogs, and Spark cluster.

2. Upload the raw datasets if the facilitator has not preloaded them.

- Create or verify the object storage folders: - `mpha/raw/` - `mpha/raw_json/` - `mpha/raw_spatial/` - `mpha/documents/` - Upload the five CSV files, one JSONL file, one GeoJSON file, and one DOCX file.

3. Open the Bronze notebook in the `01_bronze` folder and attach the shared Spark cluster.

4. Use notebooks/`aidp_bronze_pyspark.py` as the starter notebook and set:

- `raw_base` - `raw_json_base` - `raw_spatial_base` - `document_base` - `bronze_base`

5. Read each raw source and confirm the row counts match `manifest.json`.

Expected Outcome

You have an AIDP environment that is ready for the workshop: workspace, cluster, catalogs, volumes, uploaded healthcare sources, and a connected Bronze notebook.

Reference blog for the Lab 1 setup pattern:

- [Continuing Your Oracle AI Data Platform Journey: Quick Start Guide](<https://blogs.oracle.com/ai-data-platform/continuing-your-oracle-ai-data-platform-journey-quick-start-guide>)

Lab 2 - Build Bronze and Silver Layers with AI Data Platform

Objectives

- Convert raw CSV files into Bronze Delta tables without changing source values.
- Add ingestion metadata such as source file, batch id, and load timestamp.
- Create Silver Delta tables with clean, typed, conformed healthcare data.
- Add derived measures for access risk, wait-time variance, high occupancy, immunization completion, public-health pressure, triage totals, and capacity pressure bands.
- Extract spatial features from GeoJSON and prepare a spatial access insight.
- Prepare playbook document chunks for vector search and grounded chat.

Admin Notes

Participants: you can ignore this unless the facilitator asks you to use the legacy combined notebook.

1. Keep notebooks/aidp_refinement_pyspark.py only as a legacy combined reference if a facilitator wants one file instead of the split flow.

Participant Steps

1. Run notebooks/aidp_bronze_pyspark.py to write raw-preserving Bronze Delta tables.
2. Run notebooks/aidp_silver_pyspark.py to type, validate, and conform the healthcare data into Silver Delta tables.

Bronze Layer in AIDP

Bronze is the repeatable raw-data layer. It preserves the workshop source data and adds only technical ingestion metadata.

Bronze table	Source file	Grain	Bronze logic
bronze_district_health_profile	district_health_profile.csv	District	Preserve district population, need, and health-context attributes.
bronze_facility_provider_master	facility_provider_master.csv	Facility/provider	Preserve facility master data plus provider accreditation fields.
bronze_facility_operations_daily	facility_operations_daily.csv	Facility-day	Preserve daily operations, access, staffing, satisfaction, and daily quality-event measures.
bronze_population_health_weekly	population_health_weekly.csv	District-week-age group	Preserve immunization and respiratory surveillance measures in one weekly population-health source.
bronze_claims_membership_disbursement	claims_membership_disbursement.csv	Claim transaction	Preserve synthetic claim, member segment, eligibility, program, disbursement, and payment fields.
bronze_facility_capacity_events	facility_capacity_events.jsonl	Facility-event	Preserve nested JSON capacity-event payloads and add ingestion metadata.
bronze_healthcare_service_areas_geojson	healthcare_service_areas.geojson	GeoJSON FeatureCollection	Preserve district boundary, facility point, and facility catchment features in one spatial source.
Document source	MPHA_Winter_Respiratory_Response_Playbook.docx	Document	Use AIDP/document tooling to chunk and vectorize the single playbook document.

Silver Layer in AIDP

Silver is the conformed analytics layer. It applies business quality rules while remaining detailed enough for reuse.

Silver table	Grain	Silver logic
silver_district	District	Type district profile fields and deduplicate district keys.
silver_facility_provider	Facility/provider	Join facility-provider master to district profile; standardize facility type and accreditation status.

Silver table	Grain	Silver logic
silver_facility_day	Facility-day	Parse service_date, validate volumes/rates, calculate wait variance, high-occupancy flag, access risk, and quality-event measures.
silver_population_health_week	District-week-age group	Parse week_start_date, validate campaign and surveillance measures, calculate completion and no-show rates.
silver_district_health_week	District-week	Aggregate population-health rows to district-week and calculate public-health pressure index.
silver_claims_membership_disbursement	Claim transaction	Parse claim, eligibility, renewal, and disbursement dates; validate amounts; standardize claim and payment status.
silver_provider_accreditation	Provider accreditation record	Derived from silver_facility_provider for provider oversight and accreditation-risk facts.
silver_facility_capacity_event	Facility-event	Parse event timestamp, type nested JSON triage fields, calculate triage total, supply-alert count, and pressure band.
silver_spatial_feature	Spatial feature	Explode the single GeoJSON file and classify district, facility, and catchment features by source_layer.
silver_playbook_chunk	Document chunk	Created from the single DOCX playbook during the document-vector step.

Shared Bronze-to-Silver teaching pattern

Use the Bronze-to-Silver flow as the common modeling foundation before the workshop branches:

- bronze_claims_membership_disbursement -> silver_claims_membership_disbursement
- Parse service_date, eligibility_start_date, eligibility_end_date, renewal_due_date, and disbursement_date - Cast financial fields to numeric types - Standardize claim, payment, and eligibility status values
- bronze_facility_operations_daily + silver_facility_provider -> silver_facility_day
- Derive ed_wait_variance_minutes - Derive outpatient_wait_variance_minutes - Derive high_occupancy_flag - Derive access_risk_score
- bronze_population_health_weekly -> silver_population_health_week
- Derive completion_rate - Derive immunization_no_show_rate
- silver_population_health_week + silver_district -> silver_district_health_week
- Aggregate to district-week - Derive public_health_pressure_index
- bronze_facility_capacity_events -> silver_facility_capacity_event
- Derive triage_total - Derive supply_alert_count - Derive capacity_pressure_band
- bronze_healthcare_service_areas_geojson -> silver_spatial_feature
- Explode features - Classify source_layer
- MPHA_Winter_Respiratory_Response_Playbook.docx -> silver_playbook_chunk
- Derive document_id, chunk_id, page_number, section_title, embedding_model, and embedding_json

Gold Layer in Autonomous AI Lakehouse

Gold is the business serving layer. The recommended model is a dimensional star in Autonomous AI Lakehouse so OAC connects to shared, reusable dimensions and facts rather than notebook outputs or isolated flat files.

The package also includes flat Gold mart samples in `data/gold/` for quick validation, but the workshop execution path should use `data/gold_dimensional/` and `sql/create_ai_lakehouse_dimensional_gold_schema.sql`.

Gold branch used in the workshop

After Silver, the workshop branches into two Gold options:

1. Instructor-led path: Claims star schema

- Silver source: `silver_claims_membership_disbursement` - Gold fact: `mpha_fact_claims_monthly` - Gold dimensions: `mpha_dim_date`, `mpha_dim_district`, `mpha_dim_coverage_program`, `mpha_dim_claim_type` - Gold derivations: - aggregate claim rows to service month, district, program, and claim type - derive `claims_submitted`, `approved_claims`, `denied_claims`, and `pending_claims` - derive `total_submitted_amount`, `total_approved_amount`, and `total_paid_amount` - derive `avg_processing_days` - derive `denial_rate` - OAC outcome: instructor builds the provided Claims star schema workbook

2. DIY path: Facility Access Daily star schema

- Silver sources: `silver_facility_day`, `silver_facility_provider` - Gold fact: `mpha_fact_facility_access_daily` - Gold dimensions: `mpha_dim_date`, `mpha_dim_facility`, `mpha_dim_district`, `mpha_dim_pressure_band` - Gold derivations: - publish facility-day operational measures - derive or assign `pressure_band_key` - retain `district_key` in the fact so OAC joins directly to district and facility dimensions - OAC outcome: participants design their own Facility Access Daily dashboard as the challenge exercise

Gold Dimensions

Dimension	Grain	Purpose
<code>mpha_dim_date</code>	Date	Shared service date, week start, month, quarter, and year attributes.
<code>mpha_dim_district</code>	District	District population, deprivation, elderly share, chronic-condition share, and income context.
<code>mpha_dim_facility</code>	Facility	Facility master data, district identifiers, coordinates, licensed beds, and wait targets.
<code>mpha_dim_age_group</code>	Age band	Immunization age-group sorting and labels.
<code>mpha_dim_quality_event</code>	Event type and severity	Conformed quality event category and severity attributes.
<code>mpha_dim_pressure_band</code>	Pressure band	Watch, Medium, and High operating bands reused by multiple facts.
<code>mpha_dim_document_chunk</code>	Document chunk	Playbook chunk text, page, section, and embedding metadata.
<code>mpha_dim_coverage_program</code>	Coverage program	Public healthcare program, program type, and funding source.
<code>mpha_dim_claim_type</code>	Claim type	Claim type, service category, and coarse diagnosis group.
<code>mpha_dim_member_segment</code>	Member segment	Age group, risk segment, and chronic-condition flag.
<code>mpha_dim_accreditation_status</code>	Accreditation status	Accreditation body, status, level, and provider specialty scope.

Gold Facts

Fact	Grain	Main dimensions	Business purpose
<code>mpha_fact_facility_access_daily</code>	Facility-day	Date, Facility, District, Pressure Band	Daily access, wait, capacity, staffing, satisfaction, and access risk.
<code>mpha_fact_district_public_health_weekly</code>	District-week	Date, District, Pressure Band	Public-health pressure, immunization completion, no-show, positivity, and respiratory ED demand.
<code>mpha_fact_immunization_equity_weekly</code>	District-week-age group	Date, District, Age Group	Campaign reach and equity analysis by district and age group.
<code>mpha_fact_quality_event_summary</code>	Facility-event type-severity	Facility, District, Quality Event	Service-quality volume, closure time, SLA breach, and readmission indicators.
<code>mpha_fact_spatial_access_insight</code>	District	District, Pressure Band	Residents per facility, distance, pressure, and mobile-clinic planning recommendations.
<code>mpha_fact_capacity_event</code>	Facility-event	Date, Facility, District, Pressure Band	JSON capacity-event signals for real-time operations views.
<code>mpha_bridge_chat_topic_chunk</code>	Question-chunk	Document Chunk	Bridge table linking chat prompts to retrieved playbook chunks.
<code>mpha_fact_claims_monthly</code>	District-month-program-claim type	Date, District, Coverage Program, Claim Type	Claims volume, approval, denial, pending, payment yield, and processing time.
<code>mpha_fact_disbursement_monthly</code>	District-month-program-payment type	Date, District, Coverage Program	Public-program disbursement count, amount, pending, failed, and payment-cycle metrics.
<code>mpha_fact_membership_snapshot</code>	District-program-member segment snapshot	Date, District, Coverage Program, Member Segment	Membership, active eligibility, renewals due, suspended members, and high-risk members.
<code>mpha_fact_provider_accreditation</code>	Facility-provider accreditation snapshot	Date, Facility, District, Accreditation Status, Pressure Band	Accreditation score, corrective actions, days to expiry, and accreditation risk band.
<code>mpha_gold_claims_denial_risk_scores</code>	District-month-program-claim type score	Date, District, Coverage Program, Claim Type	Optional ML scoring output that prioritizes likely denial hotspots for manual review.

This is a star schema because the fact tables carry the foreign keys needed for analysis directly. Facility-grain facts join straight to both `mpha_dim_facility` and `mpha_dim_district`, while district-grain facts join directly to `mpha_dim_district`.

Flat compatibility object	Grain	Business purpose
<code>gold_facility_access_daily</code>	Facility-day	Daily access, wait-time, occupancy, staffing, and satisfaction KPIs.
<code>gold_district_public_health_weekly</code>	District-week	Public-health pressure, immunization completion, no-show, positivity, and respiratory ED pressure.
<code>gold_immunization_equity_weekly</code>	District-week-age group	Campaign reach and equity analysis by district and age group.
<code>gold_quality_event_summary</code>	Facility-severity-event type	Service-quality volume, closure time, SLA breach, and avoidable readmission indicators.

Flat compatibility object	Grain	Business purpose
gold_executive_overview	Reporting period	OAC-ready executive summary metrics and risk indicators.
gold_spatial_access_insights	District	Residents per facility, average distance, pressure, and mobile-clinic planning recommendations.
gold_capacity_event_latest	Facility-event	Near-real-time capacity signals from JSON events for the OAC real-time dashboard tab.
gold_document_chat_context	Document chunk	Vector-search context for grounded chat with data and documents.
gold_claims_summary	District-month-program-claim type	Claims adjudication, denial, approval, payment amount, and processing-time summary.
gold_disbursement_summary	District-month-program-payee type	Disbursement amount, payment status, funding source, and cycle-time summary.
gold_membership_summary	District-program-member segment snapshot	Eligibility, renewal, risk-segment, and chronic-condition membership summary.
gold_provider_accreditation_summary	Facility-provider accreditation snapshot	Accreditation score, corrective actions, expiry timing, and accreditation risk summary.

Suggested Transformations

- Parse `service_date`, `event_date`, and `week_start_date` as dates.
- Join facilities to community indicators by `district_id`.
- Calculate wait-time variance against facility targets.
- Flag high occupancy when `bed_occupancy_rate` ≥ 0.90 .
- Calculate immunization completion and no-show rates.
- Combine respiratory surveillance with immunization activity by district and week.
- Calculate `public_health_pressure_index` as a composite indicator for leadership triage.
- Explode the single GeoJSON `features` array into Silver spatial features and classify rows by `source_layer`.
- Calculate `residents_per_facility` and spatial action recommendations for mobile clinic planning.
- Chunk and vectorize the single MPHA playbook document for grounded chat.
- Normalize claims by program, claim type, service category, and coarse diagnosis group; calculate denial rate, payment yield, and processing time.
- Normalize disbursements by funding source and payee type; calculate pending, failed, paid, and cycle-time metrics.
- Normalize membership by program, age group, risk segment, chronic-condition flag, and renewal status.
- Normalize provider accreditation by facility, body, status, level, score, corrective actions, and expiry risk.
- Build or stage dimensional Gold outputs under `data/gold_dimensional/`.
- Load the dimensional Gold schema in Autonomous AI Lakehouse using `sql/create_ai_lakehouse_dimensional_gold_schema.sql`.
- Use `sql/create_ai_lakehouse_gold_layer.sql` only if a facilitator wants the simpler flat-mart path for comparison.

Expected Outcome

You have Bronze and Silver Delta layers in AI Data Platform and a dimensional Gold serving layer in Autonomous AI Lakehouse, including spatial planning, real-time event, and document-chat serving objects.

Lab 3 - Instructor-Led Claims star schema and OAC Workbook

Objectives

- Connect OAC to Autonomous AI Lakehouse.
- Build a Claims star schema dataset in OAC using the dimensional Gold tables or the curated OAC claims view.
- Create a native multi-canvas OAC workbook using freeform layout and dashboard filter controls.
- Enable OAC Assistant correctly at the dataset level and use it only for claims-dataset questions.

Admin Steps

Participants: you can skip this section and join once the facilitator confirms that the shared Lakehouse and OAC setup is ready.

1. Create or confirm the shared Autonomous AI Lakehouse instance.

- In OCI, open Oracle AI Database, then Autonomous AI Database. - Click Create Autonomous AI Database if the workshop Lakehouse instance does not already exist. - Enter the display name and database name for the shared workshop instance. - Choose the workload type Lakehouse. - Select the database version, ECPU size, storage, and network access settings required for the workshop. - Wait until the instance shows as available before continuing.

2. Create the workshop users in the Autonomous AI Lakehouse UI before participants begin SQL validation or OAC connectivity checks.

- Open the target Autonomous AI Lakehouse instance and launch Database Actions. - Sign in as ADMIN. - Open the left navigation, then go to Administration -> Database Users. - Click + Create User. - Create the shared Gold-serving owner such as MPHA_GOLD_OWNER and the participant or team users such as MPHA_P01, MPHA_P02, or MPHA_TEAM1. - Assign strong temporary passwords and enable Web Access. - Set Quota on tablespace DATA: - participant quota: 1G - facilitator or shared schema quota: 2G to 5G - Open the Granted Roles tab and grant DWR0LE, plus CONNECT if your tenancy standard still expects it. - Confirm the users are REST Enabled. - Copy the Database Actions URL from the user card and share it with the participant together with the username and temporary password.

3. Create the shared OAC connection if one does not already exist.

- In OAC, go to Create -> Connection -> Oracle Autonomous AI Lakehouse. - Use TLS for wallet-free connectivity or Mutual TLS if your environment requires a wallet upload.

4. Create the shared instructor-led claims dataset.

- Create a dataset from the Lakehouse connection. - Drag mpha_fact_claims_monthly to the Join Diagram first. - Drag mpha_dim_date, mpha_dim_district, mpha_dim_coverage_program, and mpha_dim_claim_type next. - Right-click the fact table and select Preserve Grain. - Save the dataset with a clear name such as MPHA_Claims_Star_DS. - Fast-track alternative: use MPHA_OAC_STAR_CLAIMS if the team prefers a one-table dataset.

5. Enable OAC Assistant at the dataset level if it is not already enabled.

- Return to the dataset home card. - Open Actions -> Inspect -> Search. - Set Index Dataset For to Assistant and Homepage. - Click Save, then Run Now.

Participant Steps

1. Sign in to Database Actions with the assigned workshop user and run `select user from dual;` in SQL to confirm that you land in the expected schema.
 2. Run `notebooks/aidp_gold_pyspark.py` after the Silver notebook completes.
 3. Set the `silver_base` and `gold_stage_base` paths to your object storage locations.
 4. Stage Gold-serving outputs for claims, facility access, and the supporting dimensions used by both stars.
 5. Load the dimensional Gold schema with `sql/create_ai_lakehouse_dimensional_gold_schema.sql`.
- Run the DDL and load statements in Database Actions -> SQL. - Use the schema assigned for the Gold-serving exercise, such as `MPHA_GOLD_OWNER` for the instructor-led path or a team schema if you want each table of participants to load independently.
6. Open the shared claims dataset and create the workbook.
 7. Change the first canvas layout from `Auto Fit` to `Freeform` and rename `Canvas 1` to `Claims Overview`.
 8. Create calculations in `My Calculations` only if the dataset does not already expose:
 - Claim Denial Rate - Payment Yield - Approval Rate - Pending Rate
 9. Build the KPI band with native OAC tiles.
 - Drag `claims_submitted` to the canvas to create the first tile. - Drag up to four more measures onto the same tile. - Create a second tile for a sixth KPI such as `avg_processing_days`.
 10. Add a `Dashboard Filter` control with:
 - Reporting Period - District - Coverage Program - Claim Type - Funding Source
 11. Add the core visuals:
 - district claims chart - monthly paid-amount trend - district summary table
 12. Create the additional canvases in `Freeform` layout:
 - Denial Analysis - Program Performance - Claim Type Mix
 13. Use a native Map visualization for the spatial business insight by plotting district with denial rate or paid amount when district geography is available.
 14. Reopen the workbook and use `Assistant` for claims-dataset questions only.

Recommended workshop pattern:

- keep one shared Gold-serving owner such as `MPHA_GOLD_OWNER` for loading the Claims star schema, Facility Access Daily star schema, and the OAC-facing views
- create separate participant users such as `MPHA_P01` to `MPHA_P20` for hands-on SQL checks and learning exercises

Why these settings matter:

- `Web Access` allows the participant to sign in directly to Database Actions in the browser
- `DWROLE` provides the common workshop privileges including create table, create view, create session, and `DBMS_CLOUD` execution support
- `Quota on DATA` allows the user to create and load tables during workshop exercises

Reference documentation:

- [Provision an Autonomous AI Database Instance](<https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsb/autonomous-provision.html>)

- [Connect with Built-In Oracle Database Actions](https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsb/connect-database-actions.html)
- [Create and Manage Users on Autonomous AI Database](https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsb/manage-users-create.html)
- [Manage User Roles and Privileges on Autonomous AI Database](https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsb/manage-users-privileges.html)

Instructor-led Claims star schema build

Build and explain this star first:

- Fact: mpha_fact_claims_monthly
- Dimensions:

- mpha_dim_date - mpha_dim_district - mpha_dim_coverage_program - mpha_dim_claim_type

Recommended joins:

- mpha_fact_claims_monthly.service_month_date_key -> mpha_dim_date.date_key
- mpha_fact_claims_monthly.district_key -> mpha_dim_district.district_key
- mpha_fact_claims_monthly.program_key -> mpha_dim_coverage_program.program_key
- mpha_fact_claims_monthly.claim_type_key -> mpha_dim_claim_type.claim_type_key

The role-tagged Lab 3 exercise steps above are the execution sequence to follow for the native OAC workbook build.

Recommended OAC canvases for the instructor-led workbook

1. Claims Overview

- KPI tiles for claims submitted, denied claims, denial rate, submitted amount, paid amount, and average processing days - monthly paid-amount trend - district summary table

2. Denial Analysis

- denial rate by district - denial rate by coverage program - denial rate by claim type - district map for denial exposure

3. Program Performance

- submitted amount versus paid amount by coverage program - approval and pending counts - funding-source mix by coverage program - claim volume trend by service month

4. Claim Type Mix

- claim type volume - service category mix - diagnosis-group concentration - high-denial claim types

Use dashboard_spec.md and the HTML wireframe as the instructor-led reference build.

The screenshot-backed step-by-step OAC build notes are in dashboard_spec.md, including the Lakehouse connection flow, multi-table dataset setup, freeform workbook canvases, dashboard filter control, map visual, and dataset-level OAC Assistant setup.

Important boundary:

- OAC Assistant answers questions about the indexed claims dataset.
- Questions that depend on the playbook document belong in the separate Claims and Policy Copilot optional lab.

Lab 4 - DIY Facility Access Daily star schema and Participant Dashboard Challenge

Objectives

- Apply the same dimensional pattern used in the Claims star schema.
- Build the Facility Access Daily star schema without step-by-step dashboard instructions.
- Create a participant-owned OAC dashboard from the facility access fact and its dimensions.

Admin Notes

Participants: the facilitator only needs to step in here if the shared Facility Access Daily dataset or connection has already been prepared for you.

Participant Steps

1. Build the Facility Access Daily star schema using the same dimensional pattern used in the guided claims flow.
2. Use the prepared fact and dimension targets below as your design objective.
3. Create your own participant-owned OAC dashboard using the facility access fact and its dimensions.
4. Decide which filters, KPIs, and visual interactions best support an operator workflow.

DIY Build Target

- Fact: `mpha_fact_facility_access_daily`
- Dimensions:

- `mpha_dim_date` - `mpha_dim_facility` - `mpha_dim_district` - `mpha_dim_pressure_band`

Participant Challenge Prompts

- Build a facility dashboard that answers which facilities are missing wait-time targets.
- Show occupancy, staffing pressure, and access risk by facility and district.
- Decide which filters are needed for a useful operator workflow.
- Explain why the fact keeps `district_key` directly in the fact row.

Generative Analytics Prompts

Use prompts like these in OAC narrative or assistant features:

- "Summarize the top three districts with the highest public-health pressure this month."
- "Explain which facilities are most likely to need staffing support based on occupancy and overtime."
- "Identify immunization equity gaps for older residents and recommend outreach actions."
- "Describe the relationship between respiratory positivity and emergency department wait times."
- "Which districts should receive mobile clinic sessions based on spatial access and current pressure?"
- "Use the MPHA playbook to explain what action is recommended when occupancy and ED waits rise together."
- "Which coverage programs have the highest claim denial rates and what operational follow-up is needed?"
- "Which claim types take the longest to adjudicate?"

- "Which districts show the largest claims leakage between submitted and paid amounts?"

Expected Outcome

You have an instructor-led Claims star schema workbook and a participant-built Facility Access Daily dashboard challenge grounded in the same Bronze and Silver design pattern.

Optional Lab 5 - Claims Denial Risk Scoring

Use `optional_labs/claims_denial_risk_scoring.md` and `data/gold/gold_claims_denial_risk_scores.csv`.

Objectives

- Derive ML-ready features from claims, event, district, and provider-accreditation context.
- Produce a score that estimates which district-program-claim-type combinations are most likely to need manual review.
- Publish compact scoring outputs that can be added to the Claims star schema story in OAC.

Admin Preparation

Participants: you can skip this section if the facilitator has already prepared the ML folder, cluster, and source objects.

1. Confirm that the claims-serving Gold outputs, spatial insight outputs, and provider-accreditation outputs are available.
2. Confirm that the shared Spark cluster and the `04_ml` folder are ready for participant use.

Participant Steps

1. Create a Spark notebook in the `04_ml` folder and attach the workshop Spark cluster.
2. Use `notebooks/aidp_ml_claims_denial_risk_pyspark.py` as the starter ML notebook.
3. Load `gold_claims_summary`, `gold_capacity_event_latest`, `gold_provider_accreditation_summary`, and `gold_spatial_access_insights`.
4. Build the training frame at Claims star schema grain:
 - service month - district - coverage program - claim type
5. Derive scoring features such as denial rate, average processing days, payment yield, triage total, supply alert count, public health pressure index, and accreditation risk band.
6. Define the first target such as `high_denial_flag` or `denial_risk_score`.
7. Train a simple baseline model using historical months.
8. Score the newest month and create:
 - `denial_risk_score` - `likely_denial_bucket` - `review_priority`
9. Publish the result as `gold_claims_denial_risk_scores.csv` or an equivalent serving table.
10. Blend the output back into the instructor-led Claims star schema story in OAC.

Expected Outcome

You can explain how AI Data Platform notebooks can score denial risk from the curated Gold layer and show how review teams can prioritize follow-up using `denial_risk_score`, `likely_denial_bucket`, and `review_priority`.

Optional Lab 6 - Claims and Policy Copilot

Use `optional_labs/claims_policy_copilot_agent.md`, `documents/MPHA_Winter_Respiratory_Response_Playbook.docx`, and the Claims star schema Gold objects.

Objectives

- Build a copilot that can answer claims, policy, and operational follow-up questions in natural language.
- Query curated Claims star schema data with a SQL tool.
- Ground policy and operating answers with the vectorized playbook through a RAG tool.

Admin Preparation

Participants: you can skip this section if the facilitator has already prepared the claims-serving scope, the policy document location, and the Generative AI Agents compartment access.

1. Confirm the claims-serving Gold objects are queryable in Autonomous AI Lakehouse.
2. Confirm the playbook document is available in Object Storage.
3. Confirm the facilitator can access OCI Generative AI Agents in the correct compartment.

Participant Steps

1. Place the MPHA playbook in Object Storage and, if needed, prepare a PDF or text copy for ingestion.
2. Create a knowledge base in OCI Generative AI Agents using the playbook source.
3. Create the `MPHA_Claims_Policy_Copilot` agent shell.
4. Add a SQL tool that is limited to the approved claims, disbursement, membership, accreditation, and optional scoring tables.
5. Add a RAG tool connected to the playbook knowledge base.
6. Create an endpoint for interactive testing.
7. Test three question types:
 - SQL-only - policy-only - combined claims-and-policy
8. Use the copilot as a sidecar experience during the workshop and explain how it differs from OAC Assistant.

Expected Outcome

You can explain a practical agent pattern for this workshop: AI Data Platform prepares the Gold layer, OCI Generative AI Agents provides the executable SQL-plus-RAG copilot today, and the experience stays grounded in the curated claims data and the MPHA playbook.

Optional Lab 7 - Governance and Operationalization

Admin Steps

Participants: this is primarily a facilitator or platform-owner extension lab.

1. Add data quality checks for missing dates, invalid rates, and impossible occupancy values.
2. Schedule the Bronze, Silver, and Gold Spark jobs.

3. Create OAC alerts for high occupancy or high pressure index.
4. Document the synthetic-data privacy boundary for demo and training usage.

Success Criteria

You have completed the workshop when you can:

- Explain the MPHA business use case.
- Trace data from raw files to Bronze, Silver, and Gold layers.
- Explain how Claims star schema and Facility Access Daily star schema branch from the same Silver foundation.
- Analyze claims signals from the Gold schema.
- Build the instructor-led Claims star schema workbook in OAC.
- Create your own Facility Access Daily dashboard as a participant challenge.
- Explain the optional ML scoring and Claims and Policy Copilot extension patterns.